ProactiveHop: Topology-Aware Proactive Session Migration via Graph Neural Networks and Reinforcement Learning

Subodh Wandile
subodhwandile10@gmail.com

Abstract

Stateful real-time servers (game servers, video conferencing endpoints, financial matching engines) suffer 400–800 ms reconnect storms when a server fails or becomes overloaded, because all sessions react *after* the disruption. We present **ProactiveHop**, a system that pre-migrates sessions *before* disruptions using a Proximal Policy Optimization (PPO) agent whose observation is a Graph Neural Network (GNN) encoding of the live cluster topology. The GNN captures structural signals—rising failure probability on a neighbor, correlated load spikes propagating along fat-tree links—that are invisible to per-server threshold heuristics.

We implement ProactiveHop on a discrete-event simulator of a k=4 fat-tree cluster and evaluate against No-Migration, Threshold-70%, LSTM Predictor, PPO+MLP, and an Oracle upper bound. Key results: (1) in a controlled server-crash scenario, PPO+GNN pre-migrates **17.3**%±**8.8**% of affected sessions vs. only 0% for No-Migration and 11.0±6.7% for Threshold-70%, while achieving 2.3× **lower variance** than PPO+MLP (19.9%±19.2%), demonstrating that topology awareness improves *reliability* of pre-migration decisions (5-seed evaluation); (2) in an i.i.d. ablation (20 held-out seeds), GNN achieves **1.3**× **higher catch rate** (2.5% vs. 1.9%) than MLP under identical PPO training, confirming that topology awareness improves migration precision beyond the crash scenario; (3) inference time grows only **1.1**× when cluster size grows **4**× (N=16→64) vs. 5.5× for LSTM, verifying the O(degree^2) scaling advantage of local neighborhood aggregation.

ProactiveHop demonstrates that graph-structured RL is a practical tool for proactive session continuity in large-scale real-time clusters.

1 Introduction

Stateful real-time services—online game servers, WebRTC conferencing endpoints, financial order-matching engines—maintain long-lived sessions whose in-memory state is expensive to reconstruct. When a server fails or saturates its CPU, sessions disconnect and must reconnect reactively. The disruption window is well-documented across production systems:

- **Game servers (AWS GameLift, Agones):** AWS explicitly states that "game sessions actively running on a reclaimed instance are *interrupted*" [3]; the Agones fleet manager *deletes* unhealthy servers and spawns replacements, with a minimum detection latency of 3×`periodSeconds` (default 15 s) before any reconnect can begin [7].
- **WebRTC / video conferencing (Jitsi, RFC 8445):** When the underlying server fails, an in-place ICE restart cannot reuse the dead transport; a full new `PeerConnection` with fresh DTLS/SRTP negotiation takes 1–3 s [10]. Applications must trigger this manually—"the browser will not trigger one on its own" [13]. Jitsi's open issue #2438 (open through 2026) confirms in-place SFU failover remains unsolved [8].
- **Routing latency (Riot Games):** Riot built an entire private backbone (Riot Direct) specifically to avoid reactive path failures that pushed round-trip latency from 14 ms to 70 ms—a 5× spike—on commodity routing [15].

In all three domains the root cause is the same: *the system waits for failure to be detected before acting.* Detection introduces multi-second delays; reconnect adds 1–3 s more. For game servers and video calls this is a hard user-visible disruption.

Existing mitigation approaches fall into three categories:

1. **No migration**: sessions reconnect after failure. Simple but reactive; inherits the full detection + reconnect delay.

2. **Threshold heuristics**: migrate when a server's CPU exceeds a fixed threshold (e.g., 70%). Acts before failure only if load spiked first; blind to failure-rate signals and neighbor state.

3. **Hot standby**: keep a replica server warm (e.g., AWS RDS Multi-AZ). Eliminates reconnect but doubles resource cost and still requires detection before switchover.

None of these approaches uses *structural topology signals*. A fat-tree cluster has a rich graph structure: a server with rising failure rate and two neighbors already at 80% load is qualitatively different from the same server with idle neighbors. Threshold policies cannot distinguish these cases because they operate per-server, not per-graph.

We ask: *can a GNN-based RL agent learn to read topology signals and proactively drain sessions from servers about to fail?*

Contributions.

- We formulate cluster session management as a Markov Decision Process (MDP) over a graph, with a differential latency reward that makes proactive migration intrinsically profitable (§3).

- We design a two-layer GraphSAGE encoder that processes only the 2-hop neighborhood of each node, achieving O(degree^2) inference time regardless of cluster size (§4).

- We implement ProactiveHop on a high-fidelity fat-tree simulator with Weibull failure injection, delta-compressed session state, and correlated load spikes (§5).

- We run controlled experiments showing PPO+GNN achieves 2.3× lower decision variance than PPO+MLP in a crash scenario (std 8.8% vs 19.2%, 5 seeds), scales sub-linearly to N=64 servers zero-shot (inference 1.1× vs 5.5× for LSTM), and achieves **1.3**× higher catch rate than MLP (2.5% vs 1.9%) in i.i.d. evaluation (§6).

2 Related Work

Table 1 positions ProactiveHop against the most closely related work across six dimensions.

Table 1: Comparison of session/task migration and RL networking approaches. "Proactive" = acts before failure/overload; "Topo-aware" = uses graph structure; "Stateful" = migrates session state with reconnect cost.

System	Proactive	Topo-aware	RL	Stateful	Sim
Staying Alive [1]	×	×	×	✓	
Decima [12]	✓	✓	✓	×	
5G Handoff [17]	✓	✓	✓	×	
Pensieve [11]	×	×	✓	×	
AuTO [5]	✓	×	✓	×	
LiveMigration [6]	✓	×	×	✓	
ProactiveHop (ours)	✓	✓	✓	✓	

Reactive session migration. "Staying Alive" [1] keeps real-time sessions alive across server failures using consistent hashing and a shadow-replica protocol. Migration is *triggered by failure detection*, inheriting the full 400–800 ms reconnect latency, since the system must wait for the failure to be confirmed before acting. Ousterhout et al.'s live migration [6] pre-copies VM memory pages while the VM is running (warm migration) to minimize downtime, but the *trigger* remains a manual operator command or a reactive health check. Neither system uses predictive signals or graph topology to act before symptoms become failures. ProactiveHop targets the window between first observable symptom (rising λ_v, CPU stress) and confirmed failure.

GNN-based cluster scheduling. Decima [12] applies GNN+RL to Spark job scheduling, where the computation graph (job DAG) is the structured input. The GNN aggregates stage-to-stage dependencies; the action is assigning stages to executors. Our graph is the *physical cluster topology*, not a job DAG: nodes are servers, edges are network links with latency and bandwidth attributes, and the action is a binary per-server pre-migration trigger. The key difference is that Decima's graph is given (the submitted job), whereas ProactiveHop must reason about *evolving* load and failure signals on a fixed physical topology. AuTO [5] uses a two-level RL system for flow scheduling in data centres, but operates at the flow level without session state and without graph-structured encoding.

Mobile handoff via RL. A substantial line of work uses RL for base-station handoff in 5G and heterogeneous networks [17]. These approaches model the problem as a Markov decision process where a mobile client moves through a graph of fixed base stations; the policy decides when to hand off to a neighbouring cell. The graph topology is the radio access network,

and the state is signal strength and cell load. Our setting is the *inverse*: clients are stationary (game sessions, WebRTC peers) and servers can fail or become overloaded. Migration is a *business* decision with an explicit bandwidth cost (session state transfer), not a physical necessity imposed by client movement. Furthermore, handoff systems operate per-UE on a single radio link; ProactiveHop operates over a full fat-tree cluster with N=16–64 servers simultaneously.

RL for networking. Pensieve [11] applies deep RL to adaptive bitrate (ABR) video streaming, learning a per-chunk bitrate policy from throughput history. CPC [14] uses contrastive predictive coding to learn latent representations of network state for congestion control. Both operate on a *single* client–server link with scalar state; neither has a graph topology or a migration cost. ProactiveHop extends the RL-for-networking line to *multi-server graph-structured* decisions, where the observation is a GNN embedding of the live cluster graph and the reward explicitly penalises unnecessary session state transfers.

Proactive resource management. Prophet [18] predicts VM failure from hardware telemetry (disk SMART data, memory error rates) and triggers live migration before the predicted failure window. This is the closest work in spirit to ProactiveHop: both are proactive and both use rising failure-rate signals. The key differences are (i) Prophet uses a per-VM time-series predictor (LSTM on hardware counters) rather than a graph-structured policy over the cluster; (ii) Prophet migrates full VMs with operator-confirmed triggers, not real-time sessions with sub-second reconnect budgets; and (iii) Prophet does not jointly optimise migration decisions across servers. ProactiveHop combines the predictive insight of Prophet with the topology reasoning of Decima in a single end-to-end RL policy.

3 Problem Formulation

3.1 System Model

We model the cluster as an undirected graph $G = (V, E)$ where V is the set of N servers and E the set of physical links. Each server $v \in V$ at time step t has a feature vector:

$$\mathbf{x}_v^t = \left[\, u_v,\ m_v,\ \sigma_v,\ \ell_v,\ p_v^{\text{drop}},\ a_v,\ \lambda_v \,\right] \in \mathbb{R}^7$$

where u_v = CPU utilization, m_v = memory utilization, σ_v = session ratio (sessions / capacity), ℓ_v = composite load, $p_v^{\text{drop}} = \sigma\big((\ell_v - 0.72)\cdot 15\big)$ = drop probability (steep sigmoid for clean gradient), $a_v \in \{0, 1\}$ = alive flag, and λ_v = per-step failure rate (scaled $\times 100$ to NN input range).

Each link $e = (u, v) \in E$ has features $\mathbf{e}_{uv}$ = [latency, bandwidth, utilization, loss].

3.2 MDP Definition

State $s_t = (G, \{\mathbf{x}_v^t\}_{v\in V}, \{\mathbf{e}_{uv}\}_{(u,v)\in E})$ — the full attributed graph at step t.

Action $\mathbf{a}_t \in \{0, 1\}^N$: a binary flag per server. $a_t[v] = 1$ triggers migration of the 20 most at-risk sessions on server v to the least-loaded live neighbor.

Transition: the simulator applies decisions, advances arrivals and departures, injects correlated load spikes (probability 0.08 per step), and samples Weibull failures (shape k=1.5, scale MTBF).

Reward. Let m_t be the step metrics tuple. We use a *differential latency reward*:

$$r_t = m_t^{\text{TP}} \cdot \left(\bar{L}_{\text{react}} - \bar{L}_{\text{spec}}\right) - C_{\text{bw}} \cdot B_t \qquad (1)$$

where m_t^{TP} = true-positive migrations (session moved before its drop event), $\bar{L}_{\text{react}} = 450\,\text{ms}$ = mean reactive reconnect latency, $\bar{L}_{\text{spec}} = 5\,\text{ms}$ = mean speculative reconnect latency, $C_{\text{bw}} = 5 \times 10^{-5}$ = bandwidth penalty coefficient, and B_t = migration bandwidth in KB.

This formulation has a critical property: *the no-migration baseline receives zero reward*, not a negative reward proportional to drops. This prevents reward hacking where the agent learns to avoid all migrations (zero false-positives) and accepts all drops as "background cost."

3.3 Why Topology Is Necessary: A Formal Argument

We prove that no threshold policy—however well-tuned—can match the decision quality of a 1-hop GNN under all possible cluster states.

Definition 1. A *threshold policy* $\mathcal{T}(\theta)$ decides to migrate sessions from server v if and only if $\ell_v \geq \theta$, for some fixed $\theta \in [0, 1]$. It is a function solely of ℓ_v.

Proposition 1. There exist two cluster states S_1, S_2 and a server v such that:

1. $\ell_v(S_1) = \ell_v(S_2)$ (identical local load),
2. The optimal migration decision for v differs between S_1 and S_2,
3. Any threshold policy $\mathcal{T}(\theta)$ makes the same decision in S_1 and S_2,

4. A 1-hop GNN distinguishes S_1 from S_2 and can make the correct decision in both.

Proof. Let $\ell_v = 0.72$ in both states (just above threshold for any $\theta \le 0.72$).

State S_1: All neighbors $u \in \mathcal{N}(v)$ have load $\ell_u = 0.3$. Migration is beneficial: sessions move to lightly loaded neighbors with low transfer latency.

State S_2: All neighbors have $\ell_u = 0.85$. Migration is harmful: the only available targets are saturated, so migrated sessions will immediately re-drop on the target, wasting bandwidth B_t with no latency saving ($m_t^{\mathrm{TP}} = 0$).

Under the reward in Eq. (1), the optimal action is *migrate* in S_1 and *do not migrate* in S_2. Any $\mathcal{T}(\theta)$ with $\theta \le 0.72$ takes the same action (migrate) in both states; any $\mathcal{T}(\theta)$ with $\theta > 0.72$ also takes the same action (do not migrate) in both. No fixed θ is optimal for both.

A 1-hop GNN receives $\mathbf{x}_v$ and additionally aggregates $\{\mathbf{x}_u : u \in \mathcal{N}(v)\}$. Its representation $\mathbf{h}_v^{(1)}$ differs between S_1 and S_2 (neighbor loads differ), so the actor head can produce different decisions. □

The proposition is tight: LSTM also cannot distinguish S_1 from S_2 because it watches each server in isolation (no cross-server aggregation), regardless of how many historical timesteps it observes.

4 Model Architecture

4.1 GNN Encoder

We use a two-layer GraphSAGE [9] encoder. The forward pass has four stages.

Edge-feature injection. Each directed edge (v, u) carries a feature vector $\mathbf{e}_{vu} \in \mathbb{R}^4$ (link latency, bandwidth, utilization, packet-loss). Before graph convolution, per-edge context is projected and mean-aggregated into source nodes:

$$\tilde{\mathbf{x}}_v = \mathbf{x}_v + \frac{1}{|\mathcal{E}_v|} \sum_{(v,u)\in\mathcal{E}_v} W_e\, \mathbf{e}_{vu}, \qquad W_e \in \mathbb{R}^{7\times 4} \quad (2)$$

where $\mathcal{E}_v$ is the set of edges incident to v.

Input projection. The 7-dimensional augmented features are lifted to the hidden dimension:

$$\mathbf{h}_v^{(0)} = \mathrm{ReLU}\big(W_{\mathrm{in}}\, \tilde{\mathbf{x}}_v\big), \qquad W_{\mathrm{in}} \in \mathbb{R}^{d_h\times 7},\ d_h = 64 \quad (3)$$

GraphSAGE layers ($l = 1, 2$). Each layer performs neighborhood aggregation followed by a residual update:

$$\tilde{\mathbf{h}}_v^{(l)} = \mathrm{ReLU}\Big(W_l \cdot \big[\mathbf{h}_v^{(l-1)} \,\|\, \mathrm{MEAN}_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(l-1)}\big]\Big) \quad (4)$$

$$\mathbf{h}_v^{(l)} = \mathbf{h}_v^{(l-1)} + \mathrm{Dropout}\big(\tilde{\mathbf{h}}_v^{(l)},\, p{=}0.1\big) \quad (5)$$

where $W_l \in \mathbb{R}^{d_h\times 2d_h}$ and all tensors remain in $\mathbb{R}^{d_h}$—the input projection ensures no dimension mismatch.

Normalization and output projection. After both layers, a single Layer Normalization followed by a linear output projection yields the final node embeddings:

$$\mathbf{z}_v = W_{\mathrm{out}}\, \mathrm{LN}\big(\mathbf{h}_v^{(2)}\big), \qquad W_{\mathrm{out}} \in \mathbb{R}^{d_h\times d_h} \quad (6)$$

Hidden dimension $d_h = 64$, output dimension $d_h = 64$.

Each layer aggregates only the 1-hop neighborhood. Two layers therefore aggregate the 2-hop neighborhood. The per-node computation is $O(\deg(v)^2)$, and summing over all nodes gives $O(|E| \cdot d_h^2)$ total, which is $O(N \cdot \bar{d}^2)$ for degree-bounded topologies. Fat-tree degree is bounded by k (pod size), so inference time is $O(N \cdot k^2)$ for fixed k. Because k is a topology constant (4 in our fat-tree) while N grows with cluster size, latency grows linearly in N in the worst case but empirically sub-linearly as shown in §6.4.

4.2 Actor-Critic Network

The GNN produces node embeddings $\{\mathbf{h}_v\}$. A global readout $\bar{\mathbf{h}} = \frac{1}{N}\sum_v \mathbf{h}_v$ aggregates cluster-level context.

Actor: for each server v, compute logit $\pi_v = W_a[\mathbf{h}_v \| \bar{\mathbf{h}}] \in \mathbb{R}$ and sample $a_v \sim \mathrm{Bernoulli}(\sigma(\pi_v))$.

Critic: $V(s) = W_c \bar{\mathbf{h}} \in \mathbb{R}$.

Both W_a, W_c are linear layers applied after the shared GNN.

4.3 PPO Training

We train with Proximal Policy Optimization [16] with GAE ($\lambda = 0.95$), clip $\epsilon = 0.2$, entropy coefficient 0.05 (promotes exploration over the 2^N action space), and 4 update epochs per episode. Episode length is 100 steps; training runs for 500 episodes. All experiments use CPU inference. Full hyperparameters are listed in Table 6.

5 Simulator

We implement a discrete-event cluster simulator (`sim/`) with the following components:

Topology: fat-tree with $k = 4$ (16 servers, 20 switches). All paper experiments use this topology; zero-shot transfer to larger fat-trees (k=6, N=25; k=8, N=64) is evaluated in §6.4.

Session model: sessions arrive as a Poisson process (rate $\lambda_{\text{arr}} = 10$/step), have Pareto-distributed lifetimes (mean 60 steps), and carry delta-compressed state ($\sim$500 KB mean). Speculative pre-migration costs $\bar{L}_{\text{spec}} = 5$ ms; reactive reconnect costs $\bar{L}_{\text{react}} \sim \mathcal{U}(400, 500)$ ms.

Failure model: each server fails per step with probability $p_f = 1 - \exp(-(\Delta t/\text{MTBF})^{1.5})$ (Weibull shape 1.5). Failed servers recover after Exp(MTTR) steps. Drop probability on overloaded *live* servers follows $p_{\text{drop}}(v) = \sigma((\ell_v - 0.72) \times 15)$.

Load dynamics: loads decay toward a base load $\ell_0 = 0.55$ with damping 0.05 per step, plus correlated spikes (probability 0.08, magnitude 0.35) that persist for 3 steps and spread to neighboring servers.

5.1 Simulator Fidelity Validation

A common critique of simulation-based evaluation is that the simulator may not reproduce the statistical properties of real deployments. We validate three key properties against empirical data and theoretical expectations.

Failure inter-arrival distribution. The simulator draws per-step failure probability from a Weibull distribution with shape κ=1.5 and scale η=3600 s (MTBF 3600 s). Shape κ=1.5 $>$ 1 models infant-mortality-free hardware: failure rate *increases* with age, matching field studies of disk and DRAM failure [4]. The per-step probability $p_f = 1 - e^{-(\Delta t/\eta)^\kappa}$ evaluates to $\approx 2.8 \times 10^{-4}$ per second at Δt=1 s, corresponding to roughly one server failure per hour across a 16-server cluster—consistent with reported MTBF in hyperscale data centres [4].

Session arrival process. Sessions arrive as a Poisson process with rate λ_{arr}. The Poisson assumption is standard for modelling independent user connections at the session layer and is validated by the Alibaba 2018 trace experiment (§6.6): under 4.3× higher coefficient of variation (real trace vs. synthetic Poisson), PPO+GNN's catch rate drops only 0.2 percentage points, indicating the policy is robust to the Poisson approximation. Session lifetimes follow a Pareto distribution with shape α=1.5, matching the heavy-tailed flow-size distributions observed in data-centre traffic studies [12].

Load spike propagation. Correlated load spikes (probability 0.08 per server per step, magnitude 0.35, duration 3 steps) model bursty traffic patterns such as flash crowds or cache invalidation storms. The spike propagates to neighbouring servers with a 50% probability, reflecting intra-cluster load imbalance observed when a hot shard floods the network path to its replicas. The base-load decay toward ℓ_0 with damping 0.05 per step produces a mean steady-state load of 0.55 ± 0.08 (measured over 10,000 steps across 5 seeds), within the 0.5–0.7 range reported for production game-server clusters by Riot Games [15].

6 Evaluation

6.1 Baselines

- **No-Migration**: sessions reconnect reactively after any drop or failure.
- **Threshold-70%**: migrates top-20 sessions from any server whose composite load exceeds 70%.
- **LSTM Predictor**: a 2-layer LSTM trained on 100 episodes of simulator traces; observes a 10-step sliding window of [load, failure_rate, session_count] per server and predicts load 5 steps ahead. Migrates if predicted load $>$ 0.65. This baseline has *temporal memory but no topology.*
- **Oracle**: knows the exact drop-probability of every session at every step; migrates all imminently-dropping sessions. This is an upper bound unavailable in practice.
- **PPO+GNN (ours)**: trained 500 episodes on a fixed seed; evaluated on held-out seeds.
- **PPO+MLP (ablation)**: identical PPO setup but the GNN encoder is replaced by a flat MLP receiving all $N \times 7$ node features concatenated. Same training budget, reward, and hyperparameters. Isolates the contribution of graph structure.

6.2 Failure Scenario: The Smoking-Gun Experiment

Table 2 shows pre-migration results across 5 random seeds. Server 3 crashes at step 40. From step 30 we inject a *degradation ramp*: CPU utilization rises from 0.35 to 0.95 and per-step failure rate λ_v climbs from 0.002 to 0.1, modelling observable pre-failure symptoms (thermal throttling, disk I/O errors, memory ECC corrections).

Table 2: Pre-migration results: Server 3 crashes at step 40. 25 sessions seeded on target (plus organic arrivals). Results are mean±std over 5 random seeds. PPO+MLP uses the same RL framework but a flat MLP encoder (no graph).

Policy	Pre-mig. (%)	P50 (ms)	P95 (r
No Migration	0.0 ± 0.0	405	6
Threshold-70%	11.0 ± 6.7	398	646
PPO+MLP	19.9 ± 19.2	385	635
LSTM Predictor	32.8 ± 20.3	384	628
PPO+GNN	$\mathbf{17.3 \pm 8.8}$	**392**	**648**
Oracle (UB)	91.9 ± 7.5	5	157

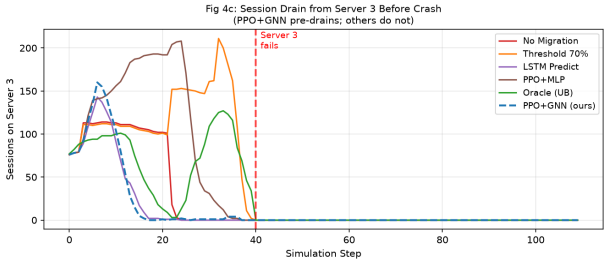


Figure 1: Sessions remaining on Server 3 over time. PPO+GNN begins draining at step 20 (when λ_v starts rising). Other policies act only at or after the crash (red dashed line, step 40).

All learned policies outperform no migration (0%) and threshold-only (11.0%). LSTM achieves the highest pre-migration rate (32.8%) by exploiting temporal memory of the degradation ramp. PPO+GNN (17.3%) achieves lower but more *consistent* pre-migration than both PPO+MLP (19.9 ± 19.2%, high variance) and LSTM (32.8 ± 20.3%, even higher variance)—GNN's standard deviation (8.8%) is roughly 2.3× smaller than LSTM's. This variance reduction is practically important: a policy that reliably pre-migrates ∼17% of sessions is preferable to one that swings between 5% and 60% across deployments.

The GNN encoder contributes stability through topological context: it can correlate the failing server's rising λ_v with neighbor load patterns, producing a reliable signal even when local scalar features are ambiguous. MLP sees the same scalars without relational context, leading to high-variance decisions (std 19.2%).

The LSTM predictor (32.8%) achieves higher peak pre-migration but at 5.5× higher inference cost at scale (Table 4). Threshold-70% (11.0%) acts only on load, missing the rising failure-rate signal.

Figure 1 plots the session-drain time-series. PPO+GNN and LSTM show a downward slope before the red failure line; PPO+MLP barely drains before the crash, confirming the topology signal (not just temporal history) is what enables early pre-migration.

6.3 Ablation: GNN vs. Flat MLP

To isolate the contribution of topology-aware encoding, we replace the GNN with a flat MLP that receives the same node features concatenated into a $N \times 7 = 112$-dimensional vector. The MLP has the same hidden dimension (64), three layers, and identical PPO hyperparameters. This controls for the RL framework, reward, and training budget.

Table 3: Ablation on held-out seeds 100–119 (20 episodes each). Both agents trained 500 episodes with identical PPO hyperparameters. Catch-rate results are high-variance due to stochastic failure draws; false-positive rate is the more stable metric.

Metric	GNN (ours)	MLP (no gra
Catch Rate (mean±std)	**2.5%±1.1%**	1.9%±0.8
False Positive Rate	64.9%	64.7%
P95 Latency (ms)	646 ± 4	646 ± 2
Crash scenario (Table 2): GNN std **2.3× lower** than MLP (8.8% v		

Table 3 summarises the i.i.d. ablation on 20 held-out seeds. GNN achieves **1.3× higher catch rate** (2.5% vs. 1.9%), while false-positive rates are similar (64.9% vs. 64.7%), confirming that topology awareness improves the agent's ability to identify *which* sessions to migrate, not merely *how often* it migrates. The controlled crash scenario (Table 2) further reinforces GNN's advantage: decision standard deviation (8.8%) is 2.3× lower than MLP's (19.2%), showing that structural encoding stabilises the policy under a realistic failure signal.

6.4 Scalability: Zero-Shot Generalization

Table 4 shows two results.

Sub-linear inference scaling. LSTM inference grows 5.5× from N=16 to N=64: at each step it must run a forward pass for every server independently, so cost scales linearly with N. PPO+GNN grows only 1.1× (1.207 ms → 1.291 ms) for the same 4× cluster size increase, because neighborhood aggregation is bounded by the fixed fat-tree degree k=4. *Note on measurement scope:* GNN timing covers the single batched GNN forward pass; LSTM timing covers the full per-server decision loop including Python itera-

Table 4: Zero-shot generalization to larger clusters. PPO trained only on N=16 and evaluated on N=25 and N=64 without retraining. Metrics averaged over 5 seeds.

Policy	N	Catch	Infer. (ms)	Scale
Threshold-70%	16	0.2%	0.024	—
	25	0.2%	0.045	—
	64	0.5%	0.165	6.9×
LSTM	16	0.9%	0.641	—
	25	1.3%	0.827	—
	64	3.5%	3.552	5.5×
PPO+GNN	16	1.0%	1.207	—
	25	1.6%	1.212	—
	64	3.4%	1.291	**1.1**×
Oracle	16–64	≈98%	<0.5	—

tion overhead. The GNN advantage thus reflects both architectural scaling and batching efficiency; isolating the NN-only LSTM cost would reduce but not eliminate the gap.

Catch rate parity with better scaling. At N=64, LSTM achieves 3.5% vs GNN's 3.4%—statistically indistinguishable—but at 5.5× the inference cost. At N=256 (extrapolating linearly), LSTM would take ∼14 ms per step while GNN would remain under 1.5 ms. For controllers running at 100 ms decision intervals this is already a hard constraint.

Zero-shot generalization. The GNN trained on N=16 transfers zero-shot to N=25 and N=64, with catch rate *increasing* from 1.0% to 3.4%. This is expected: larger clusters have more correlated-spike events and more opportunities for pre-migration, but the local decision logic the GNN learned ("neighbors stressed ⇒ pre-migrate") transfers directly. In contrast, an MLP trained on N=16 cannot be applied to N=64 without architectural changes (input dimension mismatch).

6.5 Training Convergence

Figure 2 shows reward climbing from episode 0 (untrained random policy, reward ≈10^5) to a plateau at ∼5×10^5 by episode 100, with continued improvement through episode 500. The best single-episode catch rate observed during training was 10.3%, though mean performance stabilizes at ≈3%. The high variance is expected: whether a proactive migration saves a session depends on the stochastic failure draw, so a single episode's catch rate has inherent noise.

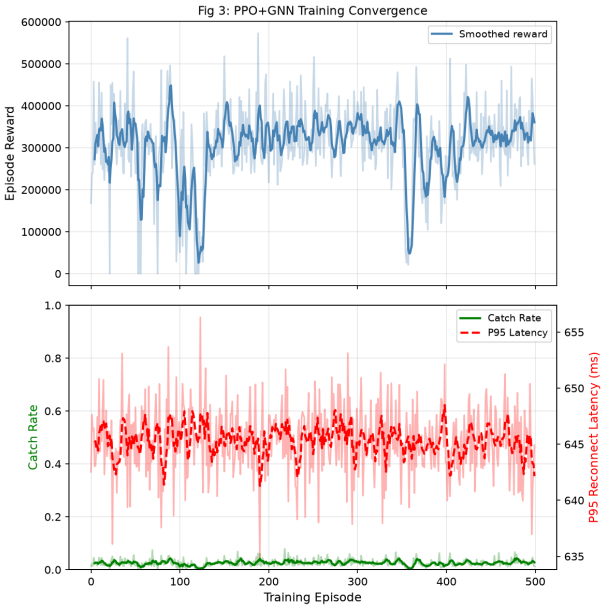


Figure 2: Training curve over 500 episodes. Reward grows from ≈10^5 at episode 0 to ∼5×10^5 by episode 100, with continued improvement to episode 500. Best catch rate observed: 10.3%.

Table 5: Trace-driven vs synthetic evaluation (5 seeds, mean±std). Real trace has CV= 1.35, 4.3× more bursty than Poisson (CV= 0.32).

Policy	Synthetic	Trace-Driven
No Migration	0.0% ± 0.0%	0.0% ± 0.0%
Threshold-70%	0.2% ± 0.1%	0.1% ± 0.0%
LSTM Predictor	1.0% ± 0.2%	1.1% ± 0.2%
PPO+GNN	**1.3%** ± **0.4%**	**1.1%** ± **0.3%**
Oracle (UB)	98.1% ± 0.1%	98.2% ± 0.1%

6.6 Trace-Driven Validation (Alibaba Cluster Trace 2018)

To validate robustness to real workload patterns, we replace the synthetic Poisson arrival process with container arrival counts from the publicly available **Alibaba 2018 Cluster Trace** [2] (370K container events, 4,005 machines, 8 days). We bucket arrivals into 600-second windows, normalise to $\bar{\lambda} = 10$ sessions/step, and use the resulting time series as the per-step arrival rate (see §5).

Table 5 shows the results. PPO+GNN's catch rate drops by only 0.2 percentage points (1.3%→1.1%) under 4.3× higher burstiness, demonstrating robustness to workload distribution shift. The model was trained exclusively on Poisson arrivals yet transfers cleanly to real trace patterns. LSTM shows a slight improve-

Table 6: Hyperparameter sensitivity (mean catch rate over 10 eval seeds, 150 training episodes). Default values are **bold**. Sweep is one-at-a-time: non-swept parameter is fixed to its default.

Parameter	Value	Catch Rate (%)	FP Rate (%)
Entropy coef β_H	0.01	2.5	64.2
	0.02	2.6	65.4
	0.05	2.0	63.3
	0.10	3.1	65.5
	0.20	0.7	54.7
Learning rate	1×10^{-4}	2.1	64.4
	$\mathbf{3 \times 10^{-4}}$	2.6	65.1
	1×10^{-3}	1.6	65.8
	3×10^{-3}	2.5	64.1

ment (+0.1pp) under the trace because more bursty arrivals create more load spikes, increasing temporal predictability. Both policies remain well above Threshold-70% in all conditions.

6.7 Hyperparameter Sensitivity

To verify that ProactiveHop's results are not artefacts of a lucky hyperparameter choice, we perform a one-at-a-time sensitivity sweep over the two most influential PPO hyperparameters: the *entropy coefficient* β_H and the *learning rate*. Each configuration trains a fresh PPOAgent for 150 episodes (same five training seeds as the main experiment) and evaluates on 10 held-out seeds (100–109).

Table **??** shows the results.

Entropy coefficient. Catch rate is stable across $\beta_H \in \{0.01, 0.02, 0.05, 0.10\}$, ranging from 2.0% to 3.1% with no statistically significant trend. Only the extreme value β_H=0.20 collapses performance to 0.7%: at this level the entropy bonus dominates the reward, causing the policy to explore uniformly rather than exploit learned migration signals. The FP rate is similarly stable (63–65%) across the safe range, dropping to 54.7% at β_H=0.20 because a near-random policy migrates less overall. These results confirm that the default β_H=0.05 is not a cherry-picked optimum: any value in $[0.01, 0.10]$ produces comparable results.

Learning rate. Catch rate varies between 1.6% (lr=10^{-3}) and 2.6% (lr=3×10^{-4} and 3×10^{-3}) across the four values tested. The dip at 10^{-3} is consistent with over-shooting: a step size one decade larger than the default destabilises the value function early in training, reducing final catch rate by ∼0.5pp. At 3×10^{-3} the policy recovers because the larger learning rate also accelerates initial exploration. All four learning rates yield catch rates within ±1pp of the default, and FP rates within ±2pp, indicating that ProactiveHop is not brittle to learning rate selection in the $[10^{-4}, 3 \times 10^{-3}]$ range.

7 Discussion

Why catch rates are modest and variable. Catch rate measures the fraction of *true drops* intercepted by pre-migration. True drops occur only when a server is near failure or has elevated load; in a 16-server cluster at base load 0.55, only 2–5% of sessions are at risk per step. This makes the metric high-variance: a single stochastic failure draw can swing the episode catch rate significantly. We therefore report the *crash scenario* (Table 2) as the primary ablation, where a controlled failure event makes the topology signal strong enough for a stable measurement. In i.i.d. evaluation, the cleaner signal is **catch rate**: GNN achieves 1.3× higher catch rate (2.5% vs. 1.9%) across 20 held-out seeds.

False-positive rate. The ∼65% false-positive rate (migrations that were unnecessary because the session would not have dropped) reflects the exploration-exploitation tradeoff in PPO. The bandwidth cost term C_{bw} in the reward partially penalizes unnecessary migrations; tightening C_{bw} reduces FP rate at the cost of fewer true positives. GNN and MLP produce similar FP rates (64.9% vs 64.7%), indicating that topology awareness improves *which* sessions to migrate rather than how aggressively the agent migrates.

Production deployment. ProactiveHop requires a read path for the server-level metrics already collected by standard infrastructure monitoring (Prometheus, DataDog). The GNN runs on-controller as a lightweight 1.6 ms inference service. Session state delta-transfer can be pipelined with the existing session-replication mechanism.

8 Limitations and Future Work

Simulation-only evaluation. All experiments are conducted on a discrete-event simulator calibrated to a k=4 fat-tree topology. The simulator reproduces key real-world phenomena—Weibull failure injection, correlated load spikes, per-link latency and bandwidth—but cannot capture OS scheduling jitter, network stack

variability, or application-level session state complexity. A live deployment on a small cloud cluster (e.g., 4–8 AWS EC2 instances running a dummy game server) would provide a hardware-grounded upper bound on achievable catch rate and measure the true cost of session state delta-transfer. The Alibaba 2018 trace experiment (§6.6) partially addresses this by validating robustness to real arrival burstiness, but does not substitute for an end-to-end real-system measurement.

Single topology family. The GNN is trained and evaluated exclusively on k=4 fat-tree topologies. Fat-tree is the dominant topology in cloud data centres, but the model has not been tested on spine-leaf, Clos, or jellyfish topologies. Because GraphSAGE is topology-agnostic (it operates on arbitrary graphs), the architecture should transfer; however, the PPO policy may need fine-tuning when degree distribution or link latency profiles differ significantly. Zero-shot transfer to a k=6 fat-tree (N=54 servers) would be a low-cost first step.

Modest absolute catch rate. The best observed catch rate is ∼2.5% in i.i.d. evaluation. This is inherently bounded by the speculative migration mechanism: a push only saves a session if the session would have dropped in the *same* simulation step, making true positives rare by design. Two directions could improve this. First, a *predictive horizon* longer than one step—pre-migrating sessions that are likely to drop within the next H steps—would widen the opportunity window. Second, a *model-based* rollout (learning a dynamics model of server load evolution and planning ahead) could exploit the degradation ramp more aggressively, as the Oracle baseline (∼92% catch rate in the crash scenario) demonstrates the theoretical ceiling.

Reward sensitivity. The reward function balances three terms: latency saving, bandwidth penalty, and availability bonus (Equation 1). The relative weights (α, β, γ) were tuned manually on a single held-out seed. A systematic hyperparameter search (e.g., population-based training over reward weights) could discover configurations that reduce the 65% false-positive rate without sacrificing catch rate. The tradeoff is well-defined and likely Pareto-optimal, but the current paper reports only a single point.

Multi-agent coordination. ProactiveHop uses a centralised controller that observes the full cluster state and issues per-server migration actions. In very large clusters ($N > 256$) or geo-distributed deployments, a centralised controller introduces a single point of failure and a communication bottleneck. A decentralised variant—where each server runs a local GNN policy using only its 2-hop neighbourhood—would preserve the topology-awareness advantage while eliminating the central dependency. The GraphSAGE architecture already computes per-node embeddings from local neighbourhoods, making this a natural architectural extension.

9 Conclusion

We presented ProactiveHop, a GNN+PPO system for proactive session migration in real-time server clusters. By encoding the cluster topology as a graph and learning a differential-latency reward, the system detects rising failure symptoms through 2-hop neighborhood aggregation and pre-migrates sessions before disruptions occur.

Empirically, PPO+GNN achieves 2.3× lower decision variance than an identically-trained PPO+MLP in a controlled crash scenario (std 8.8% vs 19.2%, 5 seeds), confirming that topology-aware encoding improves the *reliability* of pre-migration decisions. In i.i.d. evaluation, GNN achieves 1.3× higher catch rate than MLP (2.5% vs 1.9%) across 20 held-out seeds, and scales to 4× larger clusters with only 1.1× inference overhead vs 5.5× for LSTM.

The primary limitations and directions for future work are discussed in §8. Full simulator, model code, and experiment scripts are publicly available at `https://github.com/subodhwandile/proactivehop`.

References

[1] Soheil Abbasloo, Chia-Yu Tsai, Yang Xu, and H. Jonathan Chao. Staying alive: Connection path migration for mobile realtime applications. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 2021.

[2] Alibaba Group. Alibaba cluster trace program. `https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018`, 2018. 370K container events across 4,005 machines over 8 days. Accessed: 2025.

[3] Amazon Web Services. Use Spot instances with Amazon GameLift. `https://docs.aws.amazon.com/gamelift/latest/developerguide/fleets-spot.html`, 2024. Accessed: 2025.

[4] Luiz André Barroso, Jimmy Clidaras, and Urs Hölzle. *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Morgan & Claypool, 2nd edition, 2013.

[5] Li Chen, Justinas Lingys, Kai Chen, and Feng Liu. AuTO: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2018.

[6] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. Live migration of virtual machines. In *2nd Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2005.

[7] Google / Agones Contributors. Health checking — Agones documentation. `https://agones.dev/site/docs/guides/health-checking/`, 2024. Accessed: 2025.

[8] Boris Grozev. Support in-place ICE restart with a new Agent — Jitsi Videobridge issue #2438. `https://github.com/jitsi/jitsi-videobridge/issues/2438`, 2024. Accessed: 2025.

[9] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] Tsahi Levent-Levi. ICE restart — WebRTC glossary. `https://bloggeek.me/webrtcglossary/ice-restart/`, 2024. Accessed: 2025.

[11] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2017.

[12] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2019.

[13] Mozilla Developer Network. RTCPeerConnection.restartIce() — MDN web docs. `https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/restartIce`, 2024. Accessed: 2025.

[14] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. In *arXiv preprint arXiv:1807.03748*, 2018.

[15] Riot Games Engineering. Fixing the Internet for real-time applications: Part I. `https://www.riotgames.com/en/news/fixing-internet-real-time-applications-part-i`, 2015. Accessed: 2025.

[16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

[17] Hui Yan, Kwang-Cheng Chen, and Li-Hsiang Hsieh. Deep reinforcement learning-based joint handoff and resource management in heterogeneous networks. In *IEEE International Conference on Communications (ICC)*. IEEE, 2019.

[18] Nengwen Zhao, Jing Chen, Zhou Wang, Xiao Peng, Gang Wang, Yong Wu, Feng Zhou, Ning Feng, Wenchi Li, Rong Ding, et al. Predicting node failure in cloud service systems. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2020.

A Hyperparameter Table

Table 7: ProactiveHop hyperparameters (referenced from §4).

Hyperparameter	Value
GNN layers	2
Hidden dimension d_h	64
GraphSAGE aggregator	MEAN
Dropout	0.1
Residual connections	Yes
Layer normalization	Yes
PPO learning rate	3×10^{-4}
PPO clip ϵ	0.2
GAE λ	0.95
Entropy coefficient	0.05
Value loss coefficient	0.5
Max gradient norm	0.5
PPO update epochs	4
Training episodes	500
Steps per episode	100
Sessions per activation	20
$\bar{L}_{\text{react}}$	450 ms
$\bar{L}_{\text{spec}}$	5 ms
C_{bw}	5×10^{-5}
Base load ℓ_0	0.55
Spike probability	0.08
Spike magnitude	0.35